Flash Attention, Padding Lanes, and the `lse_sum != lse_sum` Guard: A Controlled Study of the NaNs nixnan Reports in PyTorch's Flash-Attention Kernels

SC26 nixnan tutorial — `flash-attn-expt` (branch `Flash-Attn`)

October 2026

Abstract

When nixnan monitored BioMistral-7B, it reported millions of NAN, $\pm\infty$ and subnormal operands. Most of them were in `pytorch_flash::flash_fwd_kernel`, and almost none reached memory. Two explanations were on the table. One was flash attention's deliberate NAN handling, in particular the split-KV combine guard `lse_logsum = (lse_sum == 0.f || lse_sum != lse_sum) ? INFINITY : ...`. The other was that nixnan misreads BioMistral's 4-bit (bitsandbytes NF4) weights.

We built TinyGPT, a 4-layer decoder-only transformer over the digits 0–9. It is small enough that every tensor can be checked, and it runs the very same PyTorch 2.6 flash-attention binaries. We monitored it with nixnan, and we then instrumented PyTorch's own flash-attention source to dump, without nixnan, the values flash computes in every lane of its tiles. Our instrumented build is bitwise-equal to PyTorch's kernel.

The answer is neither hypothesis. The NANs nixnan counts are real values in real registers, but in *padding lanes*: tile rows and columns outside the sequence. Flash never loads or clears these lanes, so they hold the previous launch's output, which is itself NAN-laden. Flash computes on them and then discards the results. The 4-bit kernels produce no NAN at all.

The combine guard's `lse_sum != lse_sum` disjunct never fires in any of these runs. We prove that for every positive softmax scale (TinyGPT's and BioMistral's $1/\sqrt{d}$ included) it *cannot* fire, for any input, including $\pm\infty$ and NAN inputs. That depends on a codegen detail we verified in the SASS. We then construct the one situation in which it does fire: a negative softmax scale. A one-argument change to TinyGPT makes the disjunct fire 1,208 times on clean input.

Contents

1 Question and approach **3**
- 1.1 The observation . . . 3
- 1.2 Hypotheses . . . 3
- 1.3 Approach . . . 3

2 Background: PyTorch 2.6's flash-attention forward pass **3**
- 2.1 Non-split kernel `flash_fwd_kernel` . . . 3
- 2.2 Split-KV kernel and combine kernel . . . 4

3 TinyGPT: the test bed **4**

4 nixnan monitoring **5**
- 4.1 Making nixnan run on PyTorch . . . 5
- 4.2 Results . . . 5

5 PyTorch source instrumentation **6**
5.1 Build . . . 6
5.2 Hooks . . . 6
5.3 Equivalence with PyTorch's kernel . . . 7

6 The padding lanes, dumped **7**
6.1 Observations . . . 7

7 The guards, dumped: inputs A and B **8**

8 When can `lse_sum != lse_sum` fire? **9**
8.1 Analysis . . . 9
8.2 Experiments: the disjunct fires . . . 10

9 Discussion **11**

10 Reproduction **12**

A Dump record layouts (`src/instr_dump.h`) **12**

B Per-call padding-lane summary (calls 8–11) **12**

1 Question and approach

1.1 The observation

The BioMistral slides (`flash-attn.pptx`) show nixnan monitoring BioMistral-7B, loaded in 4-bit NF4 through bitsandbytes and run through Hugging Face with PyTorch's scaled-dot-product attention (SDPA). nixnan reported millions of exceptional operands. They were dominated by `pytorch_flash::flash_fwd_kernel`, with instructions such as `HMMA.16816.F32` (operand 2), `FADD` with operands `[NaN, subnormal]`, and `FMUL`, while its memory counters showed almost no NaN stored. The slides attributed the filtering to the split-KV combine guard in flash attention.

1.2 Hypotheses

H1 (flash's guards). Flash attention lets NaN/∞ arise on purpose and filters them, for example at the combine guard `(lse_sum == 0.f || lse_sum != lse_sum)`.

H2 (4-bit). Packed 4-bit weights are misread by nixnan as FP16/FP32 operands.

H3 (padding lanes). Reading the source suggested a third candidate. Flash's tiles are fixed at 128×128 (or 64×256 for split-KV), and rows and columns outside the sequence are not always loaded or cleared. Arithmetic on those lanes can see arbitrary bit patterns that are never written back.

1.3 Approach

1. Build a transformer whose every tensor we can check (TinyGPT, §3), running the same flash binaries as BioMistral (same venv: torch 2.6.0+cu124, bitsandbytes 0.48.1).
2. Monitor it with nixnan: training with `SAMPLING=32`, and inference with full monitoring (§4).
3. Switch each suspect on and off: NF4 linears, FP16 linears, prompt length, and an external input that poisons one embedding (§4).
4. Instrument PyTorch's flash-attention *source*, verify the build is bitwise-equal to PyTorch's kernel, and dump every lane of the tiles and every guard decision (§5–§7).
5. Characterize exactly when `lse_sum != lse_sum` can fire, prove it unreachable for positive scales, and construct a case where it fires (§8).

2 Background: PyTorch 2.6's flash-attention forward pass

PyTorch vendors Tri Dao's FlashAttention-2 under `aten/src/ATen/native/transformers/cuda/flash_attn` (namespace `pytorch_flash`). SDPA with the flash backend calls `mha_fwd` in `flash_api.cpp`, which fills a `Flash_fwd_params` struct and launches one of two kernel families.

2.1 Non-split kernel `flash_fwd_kernel`

For head dimension 32 and FP16 the kernel traits are `Flash_fwd_kernel_traits<32,128,128,4>`: a tile of $k_M \times k_N = 128 \times 128$ (query rows $\times$ key columns), 4 warps (128 threads). The grid is $(\lceil s_q/128 \rceil, b, h)$. One CTA handles a 128-row block of queries for one (batch, head) and walks the key blocks from last to first. For each key block it:

1. loads Q (once), K and V into shared memory with `cp.async`;
2. computes $S = QK^\top$ with tensor-core MMAs (`HMMA.16816.F32`, FP16 inputs, FP32 accumulate) into the register fragment `acc_s`;
3. applies the mask (`mask.h`), setting columns $\geq s_k$ and causally masked entries to $-\infty$;

4. updates the online softmax (`softmax.h`): running row max m, running row sum ℓ, $P = 2^{S\sigma_2 - m\sigma_2}$ with $\sigma_2 = \sigma \log_2 e$, and rescales the output accumulator;

5. computes $O \mathrel{+}= PV$ with tensor-core MMAs.

At the end, `normalize_softmax_lse` divides O by ℓ and forms the log-sum-exp (LSE):

Listing 1: softmax.h:174–177 (the per-row guard).

```
float sum = row_sum(mi);
float inv_sum = (sum == 0.f || sum != sum) ? 1.f : 1.f / sum;
lse(mi) = (sum == 0.f || sum != sum) ? (Split ? -INFINITY : INFINITY) : row_max(mi) * softmax_scale + __logf(
    sum);
```

The epilogue stages O (converted to FP16) in shared memory *at the address of the Q tile* (`sO = make_tensor(sQ.data(), SmemLayoutO)`), and copies it to global memory only for rows $< s_q$.

What is not loaded. Two comments in `flash_fwd_kernel.h` matter here:

```
// We don't need to clear the sQ smem tiles since we'll only write out the valid outputs
instr_flash::copy<Is_even_MN, Is_even_K>(gmem_tiled_copy_QKV, tQgQ, tQsQ, tQcQ, tQpQ, binfo.actual_seqlen_q -
    m_block * kBlockM);
...
// We don't need to clear the sK smem tiles since we'll mask out the scores anyway.
```

V, by contrast, is loaded with `Clear_OOB_MN=true` (its out-of-range rows are zeroed), because a zero P entry times a garbage V entry could otherwise be $0 \cdot \text{NAN} = \text{NAN}$.

2.2 Split-KV kernel and combine kernel

For long keys and few query blocks, `set_params_splitkv` picks $n_{\text{splits}} > 1$ with an occupancy heuristic (`num_splits_heuristic`, using 2× the SM count; the RTX 3090 has 82 SMs). For head dimension ≤ 64 the split kernel uses a 64×256 tile, so splitting needs $s_k > 256$. Each split s runs `flash_fwd_splitkv_kernel` over its share of key blocks, writing an un-normalized partial output `Oaccum` and an LSE `lse_s` (the guard above with `Split=true`, so a zero or NAN sum gives $-\infty$). A split with no key block at all (possible under causal masking) exits early and writes $-\infty$. `flash_fwd_splitkv_combine_kernel` (16-row blocks for head dimension 32) then merges the splits:

Listing 2: flash_fwd_kernel.h:1150–1157 (PyTorch 2.6; upstream line 1204): the combine guard.

```
lse_max = lse_max == -INFINITY ? 0.0f : lse_max;  // In case all local LSEs are -inf
float lse_sum = expf(lse_accum(0) - lse_max);
for (int l = 1; l < kNLsePerThread; ++l) { lse_sum += expf(lse_accum(l) - lse_max); }
lse_sum = Allreduce<kRowsPerLoadTranspose>::run(lse_sum, sum_op);
// For the case where all local lse == -INFINITY, we want to set lse_logsum to INFINITY. Otherwise
// lse_logsum is log(0.0) = -INFINITY and we get NaN when we do lse_accum(l) - lse_logsum.
ElementAccum lse_logsum = (lse_sum == 0.f || lse_sum != lse_sum) ? INFINITY : logf(lse_sum) + lse_max;
```

The scales $e^{\text{lse}_s - \text{lse_logsum}}$ go to shared memory, and the output is $O = \sum_s e^{\text{lse}_s - \text{lse_logsum}} O_s^{\text{accum}}$. The source comment explains the `lse_sum == 0` disjunct (all splits $-\infty$). It gives no case for `lse_sum != lse_sum`; §8 shows why.

3 TinyGPT: the test bed

The completion of `312` makes three forward passes (a prefill of 4 tokens, BOS+312, then two decode steps), each with 4 attention calls: 12 flash calls in all. In every one, $s_q \in \{1, 4\}$ and $s_k \in \{4, 5, 6\}$ fall inside a single 128×128 tile, so more than 99% of the tile lies outside the sequence.

For the combine guard we need $s_k > 256$. `experiment_lse.py` prepends a fixed 297-digit context to `312`: BOS + 300 digits = 301 keys, so 2 splits for every call (prefill and decode alike). The *external input* is a vector added to the token embedding of the last prompt position, the kind a vision encoder or soft prompt supplies. Input A is all zeros. Input B has $+\infty$ in coordinate 0, standing in for an upstream FP16 overflow.

tokens	0–9 (digits), BOS = 10, EOS = 11 (masked during completion)
training set	`training-set.txt`: 100 random digit words, length 1–20 (seed 2026); none starts with 312
architecture	4 pre-LN blocks, $d = 128$, 4 heads of dimension 32, RoPE, MLP 4×, no biases; 791,808 parameters
embedding	computed as `one_hot(idx) @ W` (bit-identical to a lookup; see §4.1)
precision	FP32 parameters and linears with TF32 off (SIMT FFMA, no tensor cores); attention in FP16 through `F.scaled_dot_product_attention` with the flash backend forced
training	1,500 full-batch AdamW steps (lr $3 \cdot 10^{-3}$, cosine); loss 0.3939 vs. exact optimum 0.3938; next-token accuracy 0.8353 (optimal: the first digit of a word is unpredictable)
evaluation	greedy completion of `312` to 6 digits with a KV cache: `312452` (NF4 linears: `312428`)

Table 1: TinyGPT. Flash exists only for FP16/BF16 on tensor cores, so attention is the one place that uses HMMA.

4 nixnan monitoring

4.1 Making nixnan run on PyTorch

The first direct runs never got past start-up (20+ minutes, 27 GB RSS). There were three causes:

- **Instrumentation cost.** NVBit spends about 30 s per new cuBLAS/CUTLASS kernel. All runs therefore use `FUNCTION_WHITELIST`, which matches the *full demangled* kernel name. The full-tile kernel ($T = 128$) is a different template instance (`Is_even_MN=true`) and must be listed separately. `run_all.sh` first runs each target with a dummy whitelist to log the exact names.
- **Hangs.** `F.embedding` hangs NVBit at load time even when it is not instrumented, and so does the CUB radix sort in the backward pass of `weight[idx]`. The one-hot matmul embedding avoids both.
- **`LD_PRELOAD` over `timeout`.** If `LD_PRELOAD` also covers the `timeout` wrapper, both processes write the same `LOGFILE`. The fix is `timeout N env LD_PRELOAD=... python`.

4.2 Results

run	`SAMPLING`	result
training (1,500 steps, 57 min)	32	`flash_fwd_kernel`: 1,493 sites (2,194 NaN reports); `flash_bwd`: 458 sites, no NaN
completion `312`→6	0	`flash_fwd_kernel`: 1,367 NaN, 57 $+\infty$, 1,864 $-\infty$, 461 subnormal reports
NF4 linears	0	bitsandbytes kernels: 0 NaN/Inf, 19 subnormal sites; flash NaN unchanged in kind
FP16 linears	0	cuBLAS/CUTLASS: 41 subnormal reports at 8 sites, 0 NaN/Inf
combine A / B	0	see Table 3
prompt-length sweep	0	$T \le 16$: 300–930 NaN sites; $T = 32, 64, 96, 120, 127, 128$: 0

Table 2: nixnan runs (logs in `flash-attn-expt/logs/flash/`). The training report totals: FP32 NaN 1,490 sites (78,993,020 repeats), FP32 $-\infty$ 1,909 sites (495,634,629 repeats), FP16 NaN 704 sites (14,565,356 repeats).

N1. Clean data reproduces BioMistral's pattern. During the completion every real tensor is finite (host check), yet nixnan reports 1,183 FP32 + 184 FP16 NaN sites in `flash_fwd_kernel`, in the same instructions as the slide.

N2. The counts look like uninitialized memory. With one tile, prompt length decides: $T \le 16$ gives 300–930 sites, $T \ge 32$ gives none, and the middle is irregular ($T = 28$ has NaN, $T = 17$–24 has none). Repeating $T = 16$ gives 681, 930 and 937 sites.

N3. 4-bit is not the source. With NF4 linears and all of bitsandbytes' kernels instrumented (`kQuantizeBlockwise`, `kDequantizeBlockwise`, `kgemm_4bit_inference_naive`), there is no NaN or Inf, only 19 subnormal sites. nixnan checks FP instructions, while the packed nibbles travel through integer and shared-memory ops.

N4. The guard fires through `lse_sum == 0` only (Table 3).

N5. The combine kernel needs more than one key block, i.e. $s_k > 256$ for head dimension ≤ 64 and $s_k > 128$ for Mistral's 128. The slide's kernel is the non-split one.

N6. A NaN breaks causality through tiling. In B only position 300 is poisoned, yet rows 256–299 become NaN in layer 0 (45/301 rows). They share the second split with the poisoned key, whose V enters $P \cdot V$ as $0 \cdot \text{NaN}$.

N7. nixnan's memory check is blind here. With `INSTR_MEM=1`, every global store in the flash kernels (14 `STG.E.128`, 13 `STG.E`, 5 `STG.E.64`) is skipped as "unknown type". B writes NaN attention outputs and still reports `FP16/FP32 Memory NaN: 0`.

input	guard fired (oracle)	NaN LSE	combine LSE path (nixnan)	combine O path (nixnan)
A (zeros)	0	0	only $\pm\infty$	none
B ($+\infty$)	568 (head,row) entries	0	only $\pm\infty$	20 `FFMA` sites

Table 3: nixnan's view of the combine kernel. B's completion is `312000` (NaN logits), A's `312489`.

nixnan reports *that* NaNs occur and *where* (kernel, instruction, operand), but not in which lane, so it cannot tell a padding-lane NaN from a real one. That calls for a lane-exact view.

5 PyTorch source instrumentation

5.1 Build

`PyTorch_Instrumentation/src/` is PyTorch v2.6.0's `aten/src/ATen/native/transformers/cuda/flash_attn`, with three mechanical changes:

- the namespace is renamed `pytorch_flash`→`instr_flash`, so the extension cannot bind to libtorch's symbols;
- includes are made local;
- `flash_api.cpp` is cut after `mha_fwd`, and its dispatch is restricted to FP16/head-dim 32.

Only `flash_fwd_hdim32_fp16_sm80.cu` and `flash_fwd_split_hdim32_fp16_sm80.cu` are compiled, against the CUTLASS commit PyTorch pins (`bbe579a9`). The host side is PyTorch's own `mha_fwd`, so the split heuristic, tile sizes and template choices are SDPA's. The hooks are the hunks marked `INSTRUMENTATION` in `instrumentation.patch`; `make_src.sh` regenerates `src/` from a fresh PyTorch checkout, and we checked it reproduces `src/` byte for byte. A pybind entry (`fwd(q,k,v,causal,scale,instr)`) returns the output, the LSE, three dump tensors, and the number of splits. `instr_flash.install()` swaps it in for TinyGPT's `flash_attention`.

5.2 Hooks

`Flash_fwd_params` gains three nullable buffers (`dbg_tile`, `dbg_split`, `dbg_comb`), allocated zeroed by `mha_fwd` when dumping is on. The layout is in `src/instr_dump.h`.

`flash_fwd_kernel` For batch 0 and m-block 0 we dump the first key block processed ($n = n_{\max} - 1$, which for these sequences is the only one), whole tiles, per head:

- the shared-memory tiles `sQ`, `sK`, `sV`, read element by element through their own swizzled layouts and stored as raw FP16 bit patterns, so NaN payloads survive;
- `acc_s` before and after `apply_mask`, and `rP` after the softmax, written to their (row, col) through `thr_mma.partition_C` of an identity tensor, the same idiom the epilogue uses;

- `acc_o` after normalization;
- per row: m, ℓ, LSE, and which disjunct of the `softmax.h` guard fired.

`flash_fwd_splitkv_kernel` One record per (split, head, row) with m, ℓ, LSE, the guard outcome (none, $\ell = 0$, $\ell \neq \ell$), and a separate code for the early-exit "no key block" path.

`flash_fwd_splitkv_combine_kernel` One record per combine row, including the padding rows of the last 16-row block. It holds both split LSEs, `lse_max` (after clamping), `lse_sum`, both disjuncts, the value the guard replaced ($\log \text{lse_sum} + \text{lse_max}$), the stored value, and (via `atomicAdd`) the number of non-finite values in the combined output row.

Listing 3: The two dump helpers (src/flash_fwd_kernel.h).

```
template <typename TensorS>
__device__ __forceinline__ void instr_dump_smem(float *dst, TensorS const &s, int rows, int cols) {
    for (int i = threadIdx.x; i < rows * cols; i += blockDim.x) { dst[i] = static_cast<float>(s(i / cols, i %
     cols).raw()); }
}
template <typename TensorF, typename TensorC>
__device__ __forceinline__ void instr_dump_frag(float *dst, TensorF const &f, TensorC const &c, int ld) {
    for (int i = 0; i < size(f); ++i) { dst[get<0>(c(i)) * ld + get<1>(c(i))] = static_cast<float>(f(i)); }
}
```

5.3 Equivalence with PyTorch's kernel

`check_equivalence.py` compares output and LSE with `aten::_scaled_dot_product_flash_attention` on 8 shapes: (s_q, s_k) = (4,4,causal), (1,5), (1,6), (16,16,c), (100,100,c), (301,301,c), (1,302), (1,303), covering both the non-split and the 2-split paths. Each shape runs with dumps off and on. **All 16 comparisons are bitwise equal.** One detail matters: the build must *not* use `-use_fast_math`. With it, the combine LSE differs by 1 ulp (`logf`/`expf`), so PyTorch does not build these files with fast math. `trigger_nan_guard.py` (§8) also compares, NaN-aware, in exactly the edge cases studied below, and again everything matches. Whatever our kernels do, PyTorch's do the same.

6 The padding lanes, dumped

`dump_completion.py` runs the completion of `312` twice and dumps the last 4 flash calls of the first run: the 4 layers of the last forward pass, each a decode step with $s_q = 1$, $s_k = 6$.

6.1 Observations

P1. Every value inside the sequence is finite; every NaN is in a padding lane. Q rows $\geq s_q$ and K rows $\geq s_k$ are never loaded. The tensor-core MMA, the softmax and the PV product nonetheless run on them.

P2. The `softmax.h` guard's `sum != sum` disjunct fires, on padding rows only. 133 of the 508 padding rows of call 11 have $\ell = \text{NaN}$. Their LSE becomes $+\infty$ and their O rows stay NaN (`inv_sum` is 1 for them). Neither is stored, because of the $row < s_q$ predicates.

P3. The stale values are the previous flash call's output. 416 of the 508 stale Q rows of calls 9, 10 and 11 are bit-identical to rows of the O tile of the call just before (31 of them at the same head and row; the rest moved because the CTA ran on another SM). The epilogue staged that O (in FP16) in the same shared memory as `sQ`. Call n's 130 NaN O padding rows become call n+1's 130 (129, 133) stale Q rows with NaN. NaN propagates from launch to launch entirely inside shared memory. It never reaches global memory, and nixnan's memory counter could not see it anyway (N7).

P4. Stale K rows are zeros (404 of 488) or K rows of earlier calls (84). The 92 stale Q rows without a match among the dumped flash tiles presumably come from other kernels that ran on the same SM.

region (4 heads summed)	values	NaN	$\pm\infty$	subnormal
Q row 0 (loaded)	128	0	0	0
Q rows 1–127 (never loaded)	16,256	**3,239**	0	73
K rows 0–5 (loaded)	768	0	0	0
K rows 6–127 (never loaded)	15,616	13	0	16
V rows 6–127 (cleared)	15,616	0 (all 0)	0	0
$S = QK^\top$, valid (row 0, cols 0–5)	24	0	0	0
S, valid row, padding cols	488	13	0	0
S, padding rows	65,024	**18,274**	0	0
S after mask, padding rows	65,024	798	61,976 ($-\infty$)	0
P, padding rows	65,024	**798**	0	1
O, padding rows	16,256	**4,256**	0	0
O, row 0 (the output)	128	0	0	0
`softmax.h` guard, real rows	fired 0 of 4			
`softmax.h` guard, padding rows	fired 133 of 508, all via $\ell \neq \ell$; LSE $\to +\infty$, never stored			

Table 4: Call 11 (layer 3, last decode step), `traces/completion/REPORT.txt`. Calls 8–10 are nearly identical (3,234–3,240 NaN in stale Q, 130 guard firings each).

P5. Non-deterministic garbage, deterministic result. A second, identical run gives a different stale Q tile and a bitwise-identical real output. This is why nixnan's counts vary between repeats (N2), and why they depend irregularly on T: the leftover contents depend on which CTA last used which shared memory.

7 The guards, dumped: inputs A and B

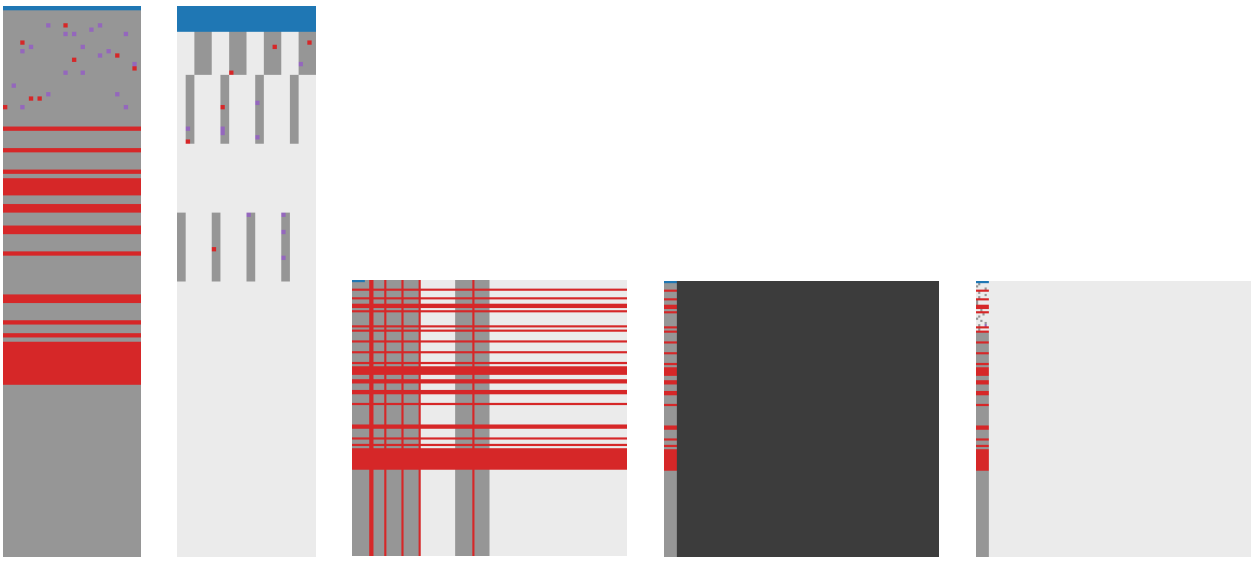
`dump_lse.py` reproduces `experiment_lse.py` (301 keys, 2 splits per call, 12 calls) with the instrumented kernels.

over all 12 calls, 4 heads	A (clean)	B ($+\infty$)
split kernel `softmax.h`, real rows: $\ell \neq \ell \to$ LSE $-\infty$	0	**1,144**
split kernel `softmax.h`, padding rows: $\ell \neq \ell$	1,588	3,931
split kernel, no key block (causal): $-\infty$ written directly	4,096	4,096
combine guard, real rows: fired	0	**568**
via `lse_sum == 0`	0	568
via `lse_sum != lse_sum`	**0**	**0**
combine guard, padding rows of the last 16-row block: fired via `lse_sum == 0`	144	144
combined output rows with non-finite values	0	**752**
completion	`...312489`	`...312000`

Table 5: Guard decisions (`traces/lse-A/`, `traces/lse-B/`). The 568 matches the earlier aten-LSE oracle exactly.

G1. `lse_sum != lse_sum` never fires; `lse_sum == 0` fires 568 times in B. The NaN is removed one level earlier. The split kernel's `softmax.h` guard maps a NaN row sum to $\text{lse}_s = -\infty$ (1,144 times on real rows in B), so the combine kernel only ever sees finite or $-\infty$ LSEs. When every split is $-\infty$, `lse_sum` is exactly 0 and the guard stores $+\infty$.

G2. The guard cleans the LSE, not the output. 752 output rows are NaN against 568 guard firings. In call 0 (layer 0, prefill) the guard fires on only 4 rows (row 300 of each head), but 180 rows are NaN: rows 256–300 of each head share the second split with the poisoned key (N6). In layer 0's decode steps split 0 is clean, so lse_sum ≥ 1 and the guard does not fire at all. The output is NaN anyway, via $0 \cdot \text{NaN}$ in $O \mathrel{+}= e^{\text{lse}_s - \text{lse_logsum}} O_s^{\text{accum}}$.



(a) sQ (b) sK (c) $S = QK^\top$ (d) S masked (e) P

Figure 1: Call 11, head 0, every element of the tile (rows = queries, top to bottom; columns = head dimension or keys). ■ finite, inside the sequence (row 0, and keys 0–5); ■ finite, padding lane; ■ zero; ■ NaN; ■ $-\infty$ (mask). The useful work is the single blue row segment at the top left. NaN rows of S come from stale Q rows, NaN columns from stale K rows. The mask removes the columns, the rows survive into P.

G3. On clean input the `softmax.h` NaN guard fires 1,588 times, all on padding rows of the split kernel's 64-row tiles: the padding-lane effect of §6 again.

G4. The combine's own padding rows (1,204 real rows fill 76 blocks of 16 = 1,216 rows) read $-\infty$ for every split, so the guard fires on them via `lse_sum == 0` in A as well as in B. nixnan's "+inf operand in `FADD R9, -R8, R9`" in run A is exactly this.

8 When can `lse_sum != lse_sum` fire?

8.1 Analysis

Write σ for the softmax scale (`params.scale_softmax`), $\sigma_2 = \mathrm{fl}(\sigma \log_2 e)$ (`scale_softmax_log2`), x for a raw FP32 score, m for a row maximum, and ℓ for a row sum. The combine computes $\mathrm{lse_sum} = \sum_s \exp(\mathrm{lse}_s - L)$ with $L = \max_s \mathrm{lse}_s$ ($-\infty \mapsto 0$). `fmaxf` ignores a NaN operand, and exp of a non-NaN argument is non-NaN, so:

Lemma 1. *`lse_sum` is NaN iff some* lse_s *is NaN, or some* $\mathrm{lse}_s = +\infty$ *(then* $L = +\infty$ *and* $\infty - \infty$*). If every* $\mathrm{lse}_s \in \mathbb{R} \cup \{-\infty\}$*, then* $\mathrm{lse_sum} \in \{0\} \cup [1, n_{\text{splits}}]$.

The split kernel writes $\mathrm{lse}_s = -\infty$ on early exit, and otherwise $\mathrm{lse}_s = (\ell = 0 \vee \ell \neq \ell)\ ?\ -\infty : m\sigma + \log \ell$. So:

Lemma 2. $\mathrm{lse}_s \in \{\mathrm{NaN}, +\infty\}$ *requires* $\ell = +\infty$*, or* $m\sigma = +\infty$ *with* ℓ *finite and positive.*

The softmax computes each term as `exp2f(x * scale - max_scaled)` with `max_scaled = (m == -inf) ? 0 : m * scale`. The source comment says this "allows the compiler to use the ffma instruction". **In the compiled code it does not.** The SASS of the split kernel (`traces/trigger/sass_scale_apply_exp2.txt`) multiplies each score by the scale parameter with `FMUL` and subtracts the scaled max with a separate `FADD` (`FADD Rx, -R47, Rx`) before the `FSETP -126` range check and `MUFU.EX2`. The term is therefore $2^{\mathrm{fl}(\mathrm{fl}(x\sigma_2) - \mathrm{fl}(m\sigma_2))}$, with two roundings. PyTorch's kernel is unfused too, as the edge case below shows.

Proposition 1 (Unreachable for $\sigma > 0$)**.** *With unfused codegen and any $\sigma > 0$, for any FP16 Q, K, V (including $\pm\infty$ and NaN),* $\mathrm{lse}_s \in \mathbb{R} \cup \{-\infty\}$ *for every split, and so* `lse_sum != lse_sum` *is false.*

Proof sketch. Fix a row. If $m = -\infty$ (every score $-\infty$, e.g. masked), then `max_scaled`$= 0$, every term is $2^{-\infty} = 0$, $\ell = 0$, and the guard gives $-\infty$. If m is NaN (every score NaN), then ℓ is NaN and the guard gives $-\infty$. Otherwise $m > -\infty$.

Case $\mathrm{fl}(m\sigma_2)$ *finite.* Rounding is monotone and $x \le m$, so $\mathrm{fl}(x\sigma_2) \le \mathrm{fl}(m\sigma_2)$. Each term is then in $[0, 1]$ (or NaN for a NaN score, which makes ℓ NaN and triggers the guard). The maximal element contributes exactly $2^0 = 1$. Across key blocks the rescale factor $2^{(m_{\mathrm{prev}} - m)\sigma_2} \le 1$. So $\ell \in [1, s_k]$. Since $\sigma < \sigma_2$, $|m\sigma| \le |m\sigma_2|$ is finite, and lse_s is finite.

Case $\mathrm{fl}(m\sigma_2) = \pm\infty$ (overflow, or $m = +\infty$). The maximal element gives $\mathrm{fl}(m\sigma_2) - \mathrm{fl}(m\sigma_2) = \infty - \infty =$ NaN, so ℓ is NaN and $\mathrm{lse}_s = -\infty$.

In every case $\mathrm{lse}_s \in \mathbb{R} \cup \{-\infty\}$, and Lemma 1 completes the proof. □

Consequences.

- TinyGPT ($\sigma = 1/\sqrt{32}$) and BioMistral ($\sigma = 1/\sqrt{128}$) can never take the NaN disjunct, whatever their inputs. Input B's $+\infty$, and any NaN, is absorbed by `softmax.h` first. This is the formal version of G1.
- With *fused* codegen (an `FFMA`, as the comment intends) the overflow case changes. If $m < 0$ and $\mathrm{fl}(m\sigma_2) = -\infty$, then $\mathrm{FFMA}(x, \sigma_2, +\infty) = +\infty$ (the exact product is finite), so $\ell = +\infty$ and $\mathrm{lse}_s = +\infty$: the disjunct becomes reachable. That needs $|m|\sigma_2 >$ FLT_MAX. With FP16 inputs $|m| \le d \cdot 65504^2 \approx 1.4 \cdot 10^{11}$ ($d = 32$), so it would take $\sigma > 1.7 \cdot 10^{27}$. With BF16 inputs, $\sigma > 1/\log_2 e \approx 0.69$ would do. With the default $1/\sqrt{d}$, $\sigma_2 < 1$ for $d \ge 3$, and even a fused build is safe. PyTorch 2.6's binary is unfused (below), so this remains hypothetical.
- **A negative scale breaks the premise** $\mathrm{fl}(x\sigma_2) \le \mathrm{fl}(m\sigma_2)$: the argument becomes $(m - x)|\sigma_2| \ge 0$. Any score spread above $128/|\sigma_2|$ overflows `exp2f` to $+\infty$. Any masked score ($x = -\infty$) gives $-\infty \cdot \sigma_2 - \mathrm{fl}(m\sigma_2) = +\infty$ exactly. Then $\ell = +\infty$, which is neither 0 nor NaN, so `softmax.h` lets it through: $\mathrm{lse}_s = m\sigma + \log\infty = +\infty$, and the combine computes $\infty - \infty$. Neither SDPA nor `mha_fwd` validates the sign of `scale`.

8.2 Experiments: the disjunct fires

`trigger_nan_guard.py` runs one decode-shaped call ($s_q = 1$, $s_k = 301$, 2 splits, 4 heads, random FP16 Q/K/V; in the large-σ cases, head 0's split-0 keys are set to $-q$ so its scores are all ≈ -37). Each case also goes through PyTorch's own kernel with the same `scale=`, and every case matches it NaN-aware.

σ	split 0 (head 0): m, ℓ, lse	split 1 (head 0)	combine `lse_sum`	disjunct
$1/\sqrt{32}$	15.23, 33.1, 6.19	17.08, 5.52, 4.73	1.23	none, 0/4
10^{30}	-37.1, 256, $-3.7 \cdot 10^{31}$	17.08, 1, $1.7 \cdot 10^{31}$	1	none, 0/4
10^{37}	-37.1, **NaN**, $-\infty$	17.08, 1, $1.7 \cdot 10^{38}$	1	none, 0/4 (but 32 NaN in O)
-0.1	15.23, 1435, 5.75	17.08, $+\infty$, $+\infty$	**NaN**	**`lse_sum != lse_sum`, 4/4**
-1	15.23, $1.9 \cdot 10^{16}$, 22.3	17.08, $+\infty$, $+\infty$	NaN	`lse_sum != lse_sum`, 4/4
-10	15.23, $+\infty$, $+\infty$	17.08, $+\infty$, $+\infty$	NaN	`lse_sum != lse_sum`, 4/4

Table 6: `traces/trigger/REPORT.txt`. At $\sigma = 10^{37}$, $m\sigma_2$ overflows to $-\infty$ and the unfused code yields $\ell =$ NaN, which `softmax.h` catches. A fused `FFMA` would have given $\ell = +\infty$ and a stored LSE of $+\infty$; PyTorch's kernel returns the same $1.708 \cdot 10^{38}$ as ours, so it is unfused too. At $\sigma = -0.1$, split 1's tile has 211 masked columns (keys 301–511 of its 256-wide block); each gives $2^{+\infty}$, so split 1's $\ell = +\infty$.

calls	combine fired	via `lse_sum == 0`	via `lse_sum != lse_sum`	rows with non-finite O
0: layer 0 prefill (301 rows/head)	1,200	0	**1,200**	1,204
1–3: layers 1–3 prefill	3,612	3,612	0	3,612
4, 8: layer 0 decode	8	0	**8**	8
5–7, 9–11: layers 1–3 decode	24	24	0	24
total (default scale: 0 / 0 / 0)	4,844	3,636	**1,208**	4,848

Table 7: TinyGPT, input A, $\sigma = -0.1$ (`traces/lse-A-scale-0.1/`). The completion becomes `...312000`.

Inside TinyGPT. One argument suffices: `dump_lse.py -input A -scale -0.1` (every attention call with $\sigma = -0.1$, clean input A).

The pattern follows the analysis exactly:

- **Layer 0, clean inputs.** The causal and padding masks provide $-\infty$ scores, which turn into $\ell = +\infty$ and $\mathrm{lse}_s = +\infty$, so `lse_sum` is NaN. Of each head's 301 rows, only row 255 escapes (4 rows in all). It sees all of split 0's keys unmasked, and split 1 has no key block for it.
- **Layers 1–3.** Their inputs are already NaN from layer 0, so `softmax.h` maps every split to $-\infty$ and the guard fires through `lse_sum == 0`, as in input B.

So the disjunct is reachable in TinyGPT, through a softmax-scale change, not through any input.

9 Discussion

What the BioMistral counts mean. nixnan checks every lane's operands, and in short sequences almost every lane of a flash tile is padding. Its NaN counts in `flash_fwd_kernel` measure padding-lane arithmetic on stale shared memory, whose contents are the previous launch's (often NaN) output. They do not measure numerical trouble in the model. The counts are non-deterministic, depend irregularly on the sequence length, and none of the values are stored. 4-bit quantization plays no part (N3). The split-KV combine guard is not where they are filtered (G1); `softmax.h` and the $row < s_q$ store predicates are.

Guards and propagation. Both guards protect the *LSE*. Neither protects the output: $0 \cdot \mathrm{NaN}$ carries a poisoned key into every row that shares its split (N6, G2), across the causal mask. A real NaN in K or V therefore corrupts more rows than causality allows, and which rows depends on the tile size.

For nixnan.

- A lane-aware filter would remove the noise: a NaN in an MMA operand that comes from rows beyond the sequence bound is expected.
- `INSTR_MEM` needs to classify packed FP16 stores (`STG.E.128` and friends); otherwise "no NaN in memory" is not evidence.
- In this tutorial the evidence needed source instrumentation, which only works when the source is available. The two methods complement each other.

Threats to validity.

- Our kernels are rebuilt from PyTorch's source for sm_86 only. They are bitwise-equal to PyTorch's binary on every case we tried, including the overflow and negative-scale edge cases, but they are not the same file.
- Head dimension 32 (TinyGPT) is not Mistral's 128: tile shapes differ (64×64 causal and 128×32 non-causal on sm_86), but the code paths, the unloaded Q/K rows, and the guards are the same.

- Stale shared-memory contents depend on scheduling, so exact counts vary run to run; conclusions P1–P5 held in every run.
- We did not re-run BioMistral itself.

10 Reproduction

All files are in `flash-attn-expt/` on branch `Flash-Attn`. Use the venv `~/repos/newmed/venv`, and set `CUDA_HOME=/usr/local/cuda-12.4` for the extension build.

```
python train.py; python generate.py --prompt 312 --length 6        # TinyGPT (tinygpt.pt is committed)
./run_all.sh                                                         # every nixnan run (Sec. 4)
cd PyTorch_Instrumentation
python check_equivalence.py        # builds the extension (~5 min), bitwise check -> traces/equivalence.txt
python dump_completion.py          # padding lanes -> traces/completion/
python dump_lse.py --input A       # guards -> traces/lse-A/   (B: traces/lse-B/)
python trigger_nan_guard.py        # lse_sum != lse_sum fires -> traces/trigger/
python dump_lse.py --input A --scale -0.1   # ... inside TinyGPT -> traces/lse-A-scale-0.1/
cd ../latex && python make_figures.py && pdflatex report && pdflatex slides
```

A Dump record layouts (`src/instr_dump.h`)

Tile dump, per head Q, K, V (128×32, raw FP16 bits); S raw, S masked, P (128×128); O (128×32); per row: m, ℓ, LSE, guard (1: $\ell = 0$, 2: ℓ NaN).

Split record (12 floats) written, split, head, row, valid ($row < s_q$), m, ℓ, LSE, guard (0 none, 1 $\ell = 0$, 2 $\ell \neq \ell$, 3 no key block).

Combine record (16 floats) written, row, valid, LSE of split 0 and 1, `lse_max`, `lse_sum`, `sum==0`, `sum!=sum`, fired, $\log \text{lse_sum} + \text{lse_max}$, stored `lse_logsum`, non-finite O count.

B Per-call padding-lane summary (calls 8–11)

call (layer)	stale-Q NaN	S pad-row NaN	P pad-row NaN	O pad-row NaN	guard on pad rows	stale Q = prev. O
8 (0)	3,240	17,919	780	4,160	130	416/508
9 (1)	3,234	17,906	780	4,160	130	416/508
10 (2)	3,234	17,911	777	4,160	130	416/508
11 (3)	3,239	18,274	798	4,256	133	416/508

(For call 8 the predecessor is call 7, which is not in the dump file. The provenance search over all earlier calls gives the same 416.)